# 精彩答疑｜建立AI原生世界世界观

Tony Bai · AI 原生开发工作流实战




你好，我是 Tony Bai。

课程结束已经一段时间了，很多同学给我留言说学到了很多，我也看到很多人在自发地推荐这门课程，非常感谢大家的支持和喜欢。随着课程的更新，我们累计了很多留言。回顾这些留言的时候，我发现了很多伙伴提出了非常有意思的问题，同时也启发了我的思考，为了让这些精彩的留言沉淀下来，这里我们专开了一个答疑章节，把大家提的好问题保留下来，希望给抱有相同问题的小伙伴一些启发。

**Q：规范和上下文怎么区分，比如团队制定的开发公约算哪种？**

A：规范本质上也是“上下文”，只是在 AI 的认知模型中扮演的角色不同。**规范是约束型上下文**，它是 AI 行为的“护栏”和“红线”。它告诉 AI “必须做什么” 和 “绝不能做什么”。比如“禁止使用 unsafe 包”、“所有 Public 函数必须有注释”。

**普通上下文则是信息型上下文**，它是 AI 完成任务所需的“素材”和“参考资料”。它告诉 AI “现有什么” 和 “怎么用”。比如 pkg/errno/code.go 的源代码、某个 API 的 JSON 结构定义。

**团队开发公约（规范）是“高权重的约束型上下文”**，它决定了 AI 产出的合规性；而代码文件是“基础的信息型上下文”，它决定了 AI 产出的正确性。两者共同构成了 AI 理解任务所需的完整语境。

**Q：SDD、spec-kit 和 claude 三者的关系是怎样的?**

A：三者可以理解为是“思想、实现与引擎”的关系。

SDD 是 “思想 / 方法论”。它告诉我们“要先写好 Spec，再生成代码，spec 是唯一的真理”。

spec-kit 是 SDD 的一个“具体实现 / 工具”。它定义了 spec/plan/tasks 的具体模板和工作流程，就像 Spring 是 IoC 思想的一种实现一样。

Claude Code 是驱动这一流程的“动力引擎”。SDD 流程中的“把 Spec 翻译成 Plan”“把 Tasks 变成 Code”，都需要 Claude 这种强大的 AI 智能体来执行。

**Q：在设计与架构中，有时候是编码与实现的时候才能发现是架构的问题。这类如何解决呢?**

A：在 SDD 工作流下，如果发现“架构问题”，可以回到 **plan.md**（技术方案）层面，告诉 AI：“这个架构在实现时遇到了 XX 问题，请修正设计。”之后，AI 会重新 update 方案（plan.md）和任务（tasks.md），然后你根据更新后的 tasks.md，让 AI 自动重写代码。

**Q：如果使用 spec 生成的代码，在测试阶段发现实现 bug，我是应该修改 spec 相关文档，让其自主修复？还是直接手动修复代码?**

A：很多使用 SDD，基于 cc 等 coding agent 的小伙伴都有类似问题。

我的想法是取决于 Bug 的性质：是“没想清楚”还是“没写对”。

如果是**“意图”错了（没想清楚）：修改 Spec**，比如你发现业务逻辑本身有漏洞，或者漏掉了某个边缘场景。这样，最好修改 spec.md，然后让 Agent 重新生成。

如果 Spec 写得很清楚，但 AI 生成的代码有逻辑错误、语法错误或性能问题，那么让 Agent 修复代码。这时 Spec 没错，你应该把错误日志、失败的测试用例反馈给 Agent，指令它：“实现有误，请根据这个错误信息修复代码”。

总之，一个原则尽量把握住：**永远维护“意图”的单一来源。**只有当 Bug 源于意图偏差时才改 Spec，否则让 Agent 去修代码。

**Q：在超过 20 人的技术团队中，目前无法把控所有开发人员都按照标准流程进行去修改 spec.md ，对于一些处理非常简单的错误，很有可能直接上手修改或者直接指示 AI 修改代码。这样一来 spec.md 就不完全正确了。**

A：SDD 在多人团队落地的确会有很多挑战！

不要指望所有小修小补都去改 Spec。毕竟“人性本懒”:)。区分一下，必须走 Spec 变更流程的，可以靠 CI/CD 流程卡点，没 Spec 不让合并。同时，对于极其简单的 Bugfix/ 小优化，也允许直接改代码，但要求 Commit Message 清晰。

另外看看是否加一个兜底方案，那就是让 AI 帮你“反向同步”。定期（或 Release 前）运行一个任务： 让 AI 读取所有新的代码变更（git diff），反向更新 spec.md。可以下达这样的指令：“根据最新的代码实现，同步更新 spec 文档，保持一致性”。可以看看效果如何。

**Q：就算在 SDD 模式下失落的翻译问题也是会存在的吧？沟通的不到位，就算有 spec.md，还是会有信息损耗的问题。意图定义这一步是为了让我们把更多的视角精力放在明确意图上，避免像之前一样只顾直接上手写代码。**

A：是的。SDD 并不能从物理上“消灭”信息损耗，而是将损耗控制在了成本最低的阶段。传统开发是“产品经理想 -> PRD -> 研发看 -> 代码”。信息损耗往往在代码写完甚至上线后才被发现，修改成本极高（重写代码、重测）。SDD 将确认环节提前到了 spec.md 阶段。在这里发现沟通不到位（损耗），修改的只是一段 Markdown 文本，而不是几千行代码。

**SDD 也可以理解为“损耗左移”。**“失落的翻译”永远存在，但 SDD 让我们在动工之前就不得不把翻译做准，而不是等到完工之后才发现翻译错了。这更像是**用“思考的成本”置换了“返工的成本”。**

**Q：我感觉 spec-kit 像是一辆“笨重的坦克”，它的执行过程非常线性。一旦进行到了后续阶段，如果发现之前的 spec 或者 plan 需要优化或补充，哪怕是很小的补充，它也会自动新建一个分支，并重新开始，导致使用起来感觉灵活性较差。另外，我非常认同 SDD 的理念，但我也好奇：通过 SDD 会生成大量的文档，当文本量很大，细节很多的情况下，AI 写出的代码能否准确地关注并执行每一个细节吗?**

A：好问题。目前的 spec-kit 作为 SDD 的参考实现，我同样感觉偏向于“全新特性开发”的线性流程，在灵活迭代上存在局限。但这是 spec-kit 的问题，并非 SDD 思想的问题。在实际操作中，我们不需要被 spec-kit 的 CLI 框死。

在本专栏后续讲解工作流时，我也没有使用 spec-kit，而是手动来实现这个流程。一旦 spec.md、plan.md、tasks.md 生成后，这些 .md 文件就是你的资产，你可以像维护代码一样，灵活地手动或让 AI 局部更新它们，而不必每次都触发“重建分支”的重型操作。

另外关于“长上下文”挑战和 AI 的注意力衰减问题，其实 SDD 的精髓在于分层：**spec.md 讲业务，plan.md 讲架构，tasks.md 讲步骤**。AI 其实不需要一次性关注所有细节。 在执行 tasks.md 中的某一个任务时（比如 Task 5），你只需要把 Task 5 的描述、相关的 spec 片段和当前涉及的代码文件喂给 AI。

目前，SDD 的工具（如 spec-kit）目前还在早期阶段，稍显笨重也是正常的。

**Q：在软件系统开发迭代过程中，尤其是有状态系统软件设计，系统积累了很久的用户数据，新功能迭代完全依赖 agent 去自动生成，会不会存在问题?**

A：对于有状态系统，新功能的迭代往往伴随着 Schema 变更和存量数据迁移。AI Agent 如果只看当前需求，很容易生成“看似完美但无法兼容旧数据”的代码，导致各种问题。

在 SDD 模式下，不能只给 Agent 新功能的 Spec，必须注入相关的上下文。比如注入 Schema， 利用 @schema.sql 让 Agent 充分理解现有数据结构。再比如告诉 Agent “数据库变更必须向后兼容”“必须生成对应的 migration 脚本”“禁止删除现有字段”等。

另外，人总是最后一道防线。**在有状态系统的迭代中，Review 和 Checkpointing 尤为重要。**对于涉及数据变更的操作，必须人工介入审查 Migration 脚本，并利用沙箱环境验证数据迁移的安全性。AI Agent 负责生成，但人需要负责为数据安全兜底。

**Q：何时该起一个新的会话？理论上可以永远就在这个会话里？**

A：理论上可行，但工程上强烈不推荐。

为什么不推荐“永远在一个会话里”呢？第一个就是你提到的 auto-summarizing，这是一个有损压缩。随着对话变长，早期的具体代码细节、特殊的约束条件会被逐渐“模糊化”甚至丢失，导致 AI 在后期变得“健忘”或不再精准。

其次是上下文污染。上一个任务的错误尝试、中间过程和临时变量，会成为下一个任务的“噪音”，干扰 AI 对新任务的判断。

此外，超长上下文的推理成本和延迟通常也会更高。

**建议遵循“一个任务，一个会话”的原则。**无论是使用 Claude Code，还是其他 coding agent 或是 web 版大模型应用也是这样。

**Q：老师好，请问一个会话怎么理解？我开发一个项目是不是都在这个会话里面，什么时候断开，还是另起一个会话重新读取原来的文件。我配置 kimi2，在会话里交互一次就要 1-2 元，消耗得飞快，是不是有问题？**

A：一个会话是从你输入 claude 启动开始，到你退出或 /clear 为止，这期间的所有对话历史（Prompt + Context）都属于一个会话。 会话越长，历史越多，每次发给 AI 的 Token 就越多（因为要把之前的对话都带上），费用也就指数级上涨。

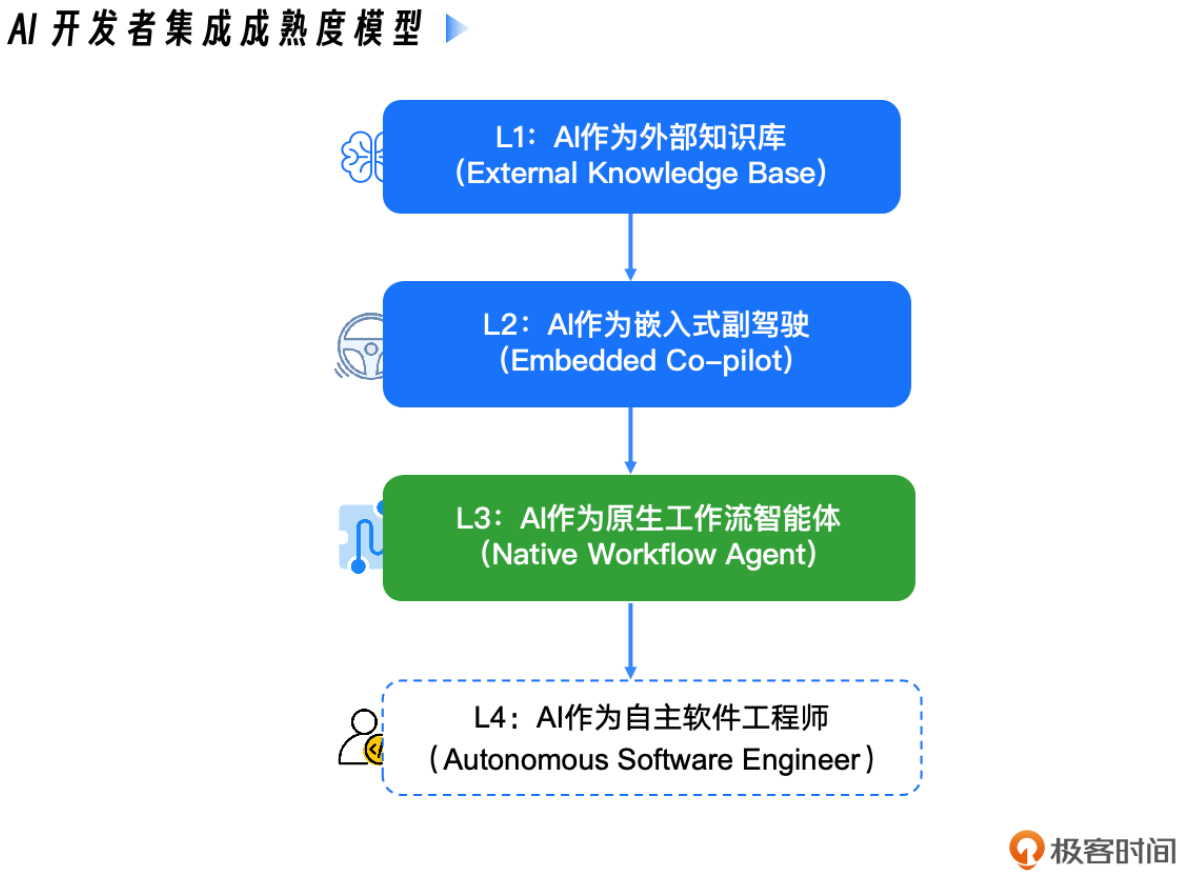
因此，考虑到上下文对 token 消耗的影响，以及上下文过大导致“幻觉”增大的可能性，一般建议：不要在一个会话里做完整个项目。遵循 “一个任务 = 一个会话”。做完一个功能点（Feature），立刻 /clear 清空历史，或重启 Claude。**会话共通的记忆，沉淀到 Claude.md 中。**

**Q：作为开发，向前一步之后与产品岗位的边界在哪儿？**

A：产品经理负责定义“模糊的业务价值”（比如用户想要什么体验？）。AI 原生工程师负责将这个模糊价值，“编译”成结构化的、无歧义的规范（spec.md）。

以前的边界是“需求文档”，现在的边界是“可执行规范”。你可能不再需要去画原型图了，但你需要**懂业务逻辑，才能写出 AI 能懂的 Spec**。你不是变成了产品经理，而是变成了“懂产品的开发人员”。

Q：Level 3 的重点在于从提问到监督的转变，可是这里面有一个很重要的前提，就是你需要有能够判断正确与否的能力，这是很多小白甚至工作多年的人不具备的，这个局要怎么破？




眼前的应对措施，不仅是“看代码”，更是“看测试”。TDD（测试驱动开发） 是小白的救命稻草。**小白需要能看懂“测试用例是否覆盖了业务场景”，这是胜任工作的底线。**从学习“如何写代码”，转向学习“如何设计测试用例”。只要你会定义“什么是对的（Test Case）”，AI 就能帮你把“对的代码”写出来。

此外，AI 时代，对于新手而言，**最危险的陷阱就是把思考的痛苦完全外包给 AI，这样就失去了最宝贵的快速成长期。因此，要避免被 AI“喂废”。**

**Q：对半路出家做开发的同学有什么建议？现在 Cursor 三下五除二代码全出来了，更没动力自学那些计算机系的基础课了。**

A：对于半路出家的同学，像 Cursor 这类 AI 工具是巨大的福音。先别纠结底层，利用 AI 快速把产品 / 功能做出来，建立信心和正反馈，这是最关键的第一步。

但接下来，就进入深水区了。AI 能“三下五除二”生成代码，但结构是否优秀，代码对错、性能优劣、安全隐患需要你来把关。换句话说，在这个 AI 时代，基础可能不再是“敲门砖”了，但它可能是你职业发展的“天花板”。

此外，有了 AI 也得继续学习，在实战中学，在 AI 的帮助下学习。比如可能不需要像以前那样枯燥地啃大部头的书籍了，把 AI 当成最好的私教。在实战中遇到不懂的，反过来补基础，效率可能会高得多。

**AI 替你由 0 到 1 搬砖，而需要你负责由 1 到 100 的设计与把关。如果你把关的底气不足，那就是学习的最好动机。**

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

## 全部留言 (9)

最新　精选



**叶绘落**
2026-01-22 来自广东

我在课程中主要学到两点:

1. Claude Code 命令行工具的深入剖析和使用技巧

2. 如何基于 SDD 思想使用 AI 进行项目的开发工作

我对 AI 开发的预期是，AI 的使用不仅仅局限于开发这个单一环节，而是功能和需求的完整交付。

所谓完整交付就是，需求文档产出、UI/UX设计、代码实现、测试、部署、项目文档迭代更新、功能交付的全流程。

也就是 AI 的使用会贯通整个互联网软件研发的全流程，以前一个团队的工作会被一个人和一堆 Agent 实现。

我目前正在设想和研究的是，为多个不同职能 Agent 配置专属 Skills ，设计工作流程让他们进行协作开发，完成项目需求的迭代和交付。

(在我写下这段留言时，大热开源项目 opencode + oh-my-opencode 已经部分实现了设想的雏形)

sepc.md 在这个过程中作用，更多是需求上的约束。

我认为如何维护项目需求文档的迭代更新也是 AI 开发很重要分一个环节。这个文档最终是要给人看的，知道一个项目的迭代历程，对于使用 AI 开发中大型项目的人来说至关重要。

我自己使用 AI 开发时会建一份 chatlog.md 的需求迭代文档，记录我每次让 AI 做需求新增、完善、变更等等信息，每一个大版本都会让 AI 结合项目现状和 chatlog.md 去更新 spec.md 。

我正在尝试的另一个解决方案，每次大版本都基于上一版本的 spec 新建一个 spec ，相当于是将 spec 当需求文档用了。

作者回复: 👍

11



**Shawna_QvQ**

2026-01-21 来自上海

十分感谢老师，之前对于AI的使用，一直是处于LEVEL2，AI是一个被动的回答者，实际高强度使用中会发现，需要在每一次和AI的对话中，重复的声明一些特定的prompt，在这个课程中发现自己在实际中使用AI的痛点得到了解答，后续应该就是先个人实践积攒经验，后续看看能否推广到项目。

作者回复: [手动抱拳]

4

**肥猫不开心**
2026-02-03 来自重庆

老师，复杂项目的1到2，如何利用ai开发呢，感觉这块很难玩起来啊

作者回复:
别指望 AI 一口气理解整个复杂项目。

存量复杂项目，只能用分而治之的策略。比如：每次迭代只聚焦一个模块。用 @module_path/ 精准注入上下文，而不是 @.（整个项目）。

不要重写 spec.md。为新功能（“从 1 到 2”的增量）新建一个 specs/feature-v2/ 目录。

复杂项目最难的是“读懂旧代码”。可以先让 AI 解释旧逻辑、画出调用链等，确认它懂了，再让它做改动。

共 3 条评论>

2

**H**
2026-01-27 来自北京

遵循“一个任务 = 一个会话”。做完一个功能点（Feature），立刻 /clear 清空历史，或重启 Claude。会话共通的记忆，沉淀到 Claude.md 中。
老师这个怎么实操呢?一个任务一个回话指的是tasks里一个小任务么比如1.1.1?

作者回复: 这里的“任务”粒度，确实需要根据实际情况灵活把握，不能教条地对应 tasks.md 里的每一个小点。

原则是： 上下文是否需要延续?

场景一（需要延续）：

如果你正在开发一个 User 模块，tasks.md 里有 1.1（建表）、1.2（写 Service）、1.3（写 API）。这三个小任务是紧密相关的，写 API 时 AI
需要知道 Service 长什么样。这时，可以在同一个会话里一口气做完这三个小任务。

场景二（可以断开）：

当你完成了 User 模块的所有开发（Feature 完成），准备开始做毫不相关的 Order 模块时。这时必须 /clear，或新开一个session会话。 因为 Order 模块不需要知道 User 模块开发过程中的纠结和错误尝试。清理上下文或新建会话，能让 AI 以更清醒的状态开始新任务。

2



**6点无痛早起学习的和...**

2026-01-27 来自北京

在实际工作中，我遇到过这样的真实问题。
我让 AI 按照 SDD 去设计的存量系统的一个老功能里面链路的迭代，最终上线了。
但现在在线上功能爆雷了。代码开发完了，并没有考虑到这个功能全链路上（不局限于我的这个服务）对端用户的体验，导致用户在端的 7s 没拿到操作的结果。
其实这个问题本质上就是说，AI 它并不知道，你这个链路上改动会影响哪些东西，所以，对于存量系统，有时候维护一个你的接口，在整个链路如何串联起来的知识库，这样，当 AI 去改动现有代码的时候，它就知道可以帮你排查出来啊，这个改动影响了什么链路。从而让咱们反向思考和意识到。

作者回复: 感谢分享。

1



**coderlee**

2026-04-21 来自福建

跟着课程完整的学了一遍，再结合公司业务线，个人理解，全新系统开发也好，还是已有系统需求迭代，还是要打造契合公司技术栈的一套sdd插件，开源的sdd工具只有借鉴意义，但是不能作为救命稻草，否则就会把利器变为毒药。本应该半小时完成的事情，硬生生拖垮成半天。

作者回复: 👍

**Geek_9ec8bf**

2026-04-05 来自广东

请问一下，在智能体开发这一块，我要如何做语言选型？到底是选Python呢，还是选TypeScript。Python的langchain跟langgraph生态发展由来已久，也很完善很成熟。TypeScript最近也非常火，特别是openclaw出来之后，它有很多杀手锏的这个应用，同时claude code也是用它来做的开发，还有open code。我们团队3个后端，两个前端，其中只有一个比较懂langchain的开发，想要问下做智能体开发，到底要如何选型？

> 作者回复: 智能体（Agent）开发的核心，不在于语言本身，而在于对大模型（LLM）能力的理解、对工作流的编排（Orchestration）以及对工具（Tools）的
> 调用和管理，以及对大模型调用行为的管控。
>
> 目前在业界，ts和python在开发agent的确有优势。openclaw, cc都是ts开发的，hermes是用python开发的。
>
> 但目前各个团队代码基本都是ai生成的，所以选择哪个语言，关键还是看你的团队是否能更好的对大模型生成的代码做好理解和review。毕竟生产出的agent是要商用的，而不仅仅是一个纯vibe coding出来的玩具级别的demo。






**InfoQ_6792a017d8d3**
2026-04-02 来自浙江

醍醐灌顶，后悔学晚了，推荐给了周围好几个要好的伙伴

> 作者回复: [手工抱拳]






**莉**
2026-02-03 来自江苏

AI 替你由 0 到 1 搬砖，而需要你负责由 1 到 100 的设计与把关。如果你把关的底气不足，那就是学习的最好动机。
说的真好！